Preprocessing excerpt (continued) *preprocessing/pingstation_preprocess.py*

```
        return int(value)

    def clean_speed_cms(self, speed):
        """Normalizes horizontal speed to centimeters/second."""
        if speed is None:
            return 0

        if isinstance(speed, str):
            text = speed.strip().lower().replace(",", "")
            if "kt" in text:
                knots = self._to_float(text.replace("kt", "").strip())
                mps = knots * 0.514444
                return int(mps * 100)
            speed = text

        value = self._to_float(speed)

        # If the value looks like m/s, convert to cm/s.
        if value < 1000:
            return int(value * 100)

        return int(value)

    def _normalize_timestamp(self, timestamp):
        if timestamp in (None, ""):
            return datetime.now(timezone.utc).isoformat()

        text = str(timestamp).strip()
        if not text:
            return datetime.now(timezone.utc).isoformat()

        try:
            return datetime.fromisoformat(text.replace("Z", "+00:00")).isoformat()
        except ValueError:
            return datetime.now(timezone.utc).isoformat()

    def extract_time_features(self, timestamp):
        try:
            dt = datetime.fromisoformat(str(timestamp).replace("Z", "+00:00"))
            return {"hour": dt.hour, "minute": dt.minute}
        except (TypeError, ValueError):
            now = datetime.now()
            return {"hour": now.hour, "minute": now.minute}

    def preprocess(self, raw_data):
        if not isinstance(raw_data, dict):
            return {
                "icaoAddress": "UNKNOWN",
                "callsign": "UNKNOWN",
                "latDD": 0.0,
                "lonDD": 0.0,
                "altitudeMM": 0,
                "headingDE2": 0,
                "horVelocityCMS": 0,
                "timestamp": datetime.now(timezone.utc).isoformat(),
                "hour": datetime.now().hour,
                "minute": datetime.now().minute,
            }

        lat = self._to_float(
```